National Telecommunication Institute (NTI)
Digital Design Using FPGA — Training Program

Final Project Report

Design and Implementation of a Single-Cycle RV32I RISC-V Processor on FPGA

1. Team Members

#	Name	Role
1	Mohammed Saeed Mohammed	Control Unit & ALU Control
2	Mohammed Awad-Allah Mohammed	ALU & Register File
3	Mario Mody Zaher	Immediate Generator & Memory Modules
4	Sagda Hossameldin Ibrahim	Top-Level Integration & PC/MUX/Branch Logic
5	Haroun Taha	Testbench, Assembly Programs, I/O & Documentation

2. Project Overview

This project implements a working central processing unit (CPU) from scratch, based on the RISC-V open instruction set architecture, using the Verilog hardware description language. The core is verified through simulation and then deployed on a physical FPGA development board for a live demonstration.

The processor follows a single-cycle design: every instruction — fetch, decode, and execute — completes within one clock cycle. This is the simplest processor architecture to design and understand, which makes it realistic to build, verify, and demo within the training's 5-day timeline while still covering all the core building blocks of a real CPU (datapath, control logic, register file, ALU, and memory).

2.1 What is RISC-V / RV32I?

RISC-V is an open, royalty-free instruction set architecture (the 'language' a processor understands) that has become a widely used industry and academic standard. RV32I is its base 32-bit integer subset: 32 means the registers and data paths are 32 bits wide, and 'I' means only the base integer instructions are included — no multiplication/division (M extension), no floating point (F extension), and no compressed instructions (C extension). This project implements a further-restricted subset of RV32I: 14 instructions across all six RISC-V instruction formats (R, I, S, B, U, J), enough to demonstrate arithmetic, memory access, and control flow.

2.2 How the Processor Works

Each clock cycle, the processor performs three stages simultaneously (not in separate cycles, since this is single-cycle):

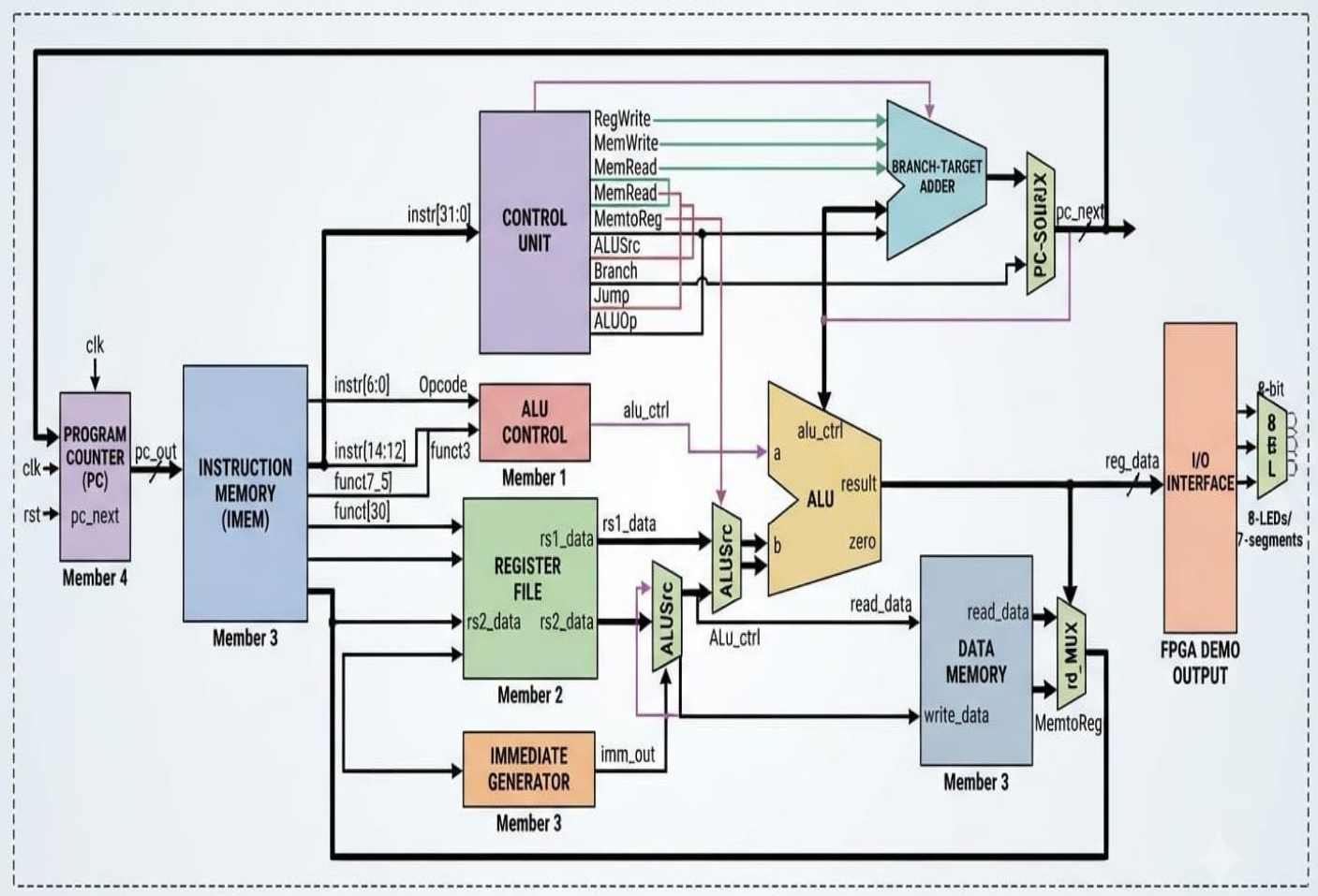
- Fetch — the Program Counter (PC) supplies an address; the Instruction Memory returns the 32-bit instruction stored there.
- Decode — the Control Unit reads the instruction's opcode (and funct3/funct7 fields) and generates the control signals that configure the rest of the datapath; the Register File supplies the operand values; the Immediate Generator extracts any constant embedded in the instruction.
- Execute — the ALU performs the arithmetic/logic operation, or the Data Memory is read/written for load/store instructions; the result is written back to the Register File, and the next PC value is computed (sequential, or a branch/jump target).

3. Supported ISA Subset (RV32I, Restricted)

Instruction	Type	Opcode	Operation
ADD	R	0110011	rd = rs1 + rs2 (funct3=000, funct7=0000000)
SUB	R	0110011	rd = rs1 − rs2 (funct3=000, funct7=0100000)
AND / OR / XOR	R	0110011	bitwise logic (funct3 = 111 / 110 / 100)
SLT	R	0110011	rd = (rs1 < rs2) ? 1 : 0 (funct3=010)
SLL / SRL / SRA	R	0110011	shifts (funct3 = 001 / 101, funct7 selects logical/arith.)
ADDI	I	0010011	rd = rs1 + imm (funct3=000)
LW	I	0000011	rd = MEM[rs1 + imm] (funct3=010)
JALR	I	1100111	rd = PC+4 ; PC = (rs1+imm) & ~1
SW	S	0100011	MEM[rs1 + imm] = rs2 (funct3=010)
BEQ	B	1100011	if (rs1==rs2) PC += imm (funct3=000)
BNE	B	1100011	if (rs1!=rs2) PC += imm (funct3=001)
LUI	U	0110111	rd = imm << 12
JAL	J	1101111	rd = PC+4 ; PC += imm

4. System Block Diagram

The diagram below shows the full single-cycle datapath: how the Program Counter, Instruction Memory, Control Unit, Register File, Immediate Generator, ALU, Data Memory, and I/O Interface are connected, with each block's owning member labeled.



5. Module Specifications & Task Ownership by Member

Each member should implement their module(s) exactly to the port lists below, so that top-level integration is a plug-and-play process.

Member 1 — Mohammed Saeed Mohammed Ahmed

Role: Control Unit & ALU Control

5.1 Control Unit

Signal	Dir	Width	Description
`opcode`	in	7	instr[6:0]
`RegWrite`	out	1	enable register file write
`MemWrite`	out	1	enable data memory write
`MemRead`	out	1	enable data memory read
`MemtoReg`	out	1	0: ALU result, 1: memory data
`ALUSrc`	out	1	0: rs2, 1: immediate
`Branch`	out	1	is a branch instruction
`Jump`	out	1	is a jump instruction (JAL/JALR)
`ALUOp`	out	2	coarse op class, feeds ALU Control

5.2 ALU Control

Signal	Dir	Width	Description
`ALUOp`	in	2	from Control Unit
`funct3`	in	3	instr[14:12]
`funct7_5`	in	1	instr[30]
`alu_ctrl`	out	4	exact ALU operation code

5.3 Tasks

- Design the opcode → control-signal truth table (Section 3 above) and implement it as a Verilog case statement.
- Implement the ALU Control mapping: ALUOp + funct3 + funct7[5] → 4-bit alu_ctrl.
- Write a standalone testbench applying every opcode and checking all control outputs.

Member 2 — Mohammed Awad-Allah Mohammed

Role: ALU & Register File

5.4 ALU

Signal	Dir	Width	Description
`a`	in	32	operand 1 (rs1_data)
`b`	in	32	operand 2 (rs2_data or imm)
`alu_ctrl`	in	4	operation select (from ALU Control)
`result`	out	32	ALU result
`zero`	out	1	1 if result == 0 (for branches)

5.5 Register File

Signal	Dir	Width	Description
`clk`	in	1	clock
`rs1_addr`	in	5	read port 1 address
`rs2_addr`	in	5	read port 2 address

rd_addr	in	5	write address
rd_data	in	32	write-back data
RegWrite	in	1	write enable
rs1_data	out	32	read data 1
rs2_data	out	32	read data 2

Note: x0 (address 0) must always read as 32'b0, regardless of writes.

5.6 Tasks

- Implement the ALU with all required operations (Section 3) and the zero flag.
- Implement the 32×32 register file, with x0 hardwired to 0, synchronous write / combinational read.
- Testbench: write/read every register, verify x0 stays 0, verify each ALU operation with edge cases (overflow, equal operands, negative numbers).

Member 3 — Mario Mody Zaher

Role: Immediate Generator & Memory Modules

5.7 Immediate Generator

Signal	Dir	Width	Description
instr	in	32	full instruction
imm_out	out	32	sign-extended immediate

5.8 Instruction Memory

Signal	Dir	Width	Description
addr	in	32	byte address (use addr[7:2] to index)
instr	out	32	fetched instruction

5.9 Data Memory

Signal	Dir	Width	Description
clk	in	1	clock
addr	in	32	byte address
write_data	in	32	data to store (SW)
MemWrite	in	1	write enable
MemRead	in	1	read enable
read_data	out	32	loaded data (LW)

5.10 Tasks

- Implement immediate generation for the I/S/B/U/J formats with correct sign-extension.
- Implement instruction memory (ROM, loaded via $readmemh) and data memory (RAM, synchronous write).
- Prepare the hex machine-code file for the test assembly programs (hand-assemble, or use a RISC-V assembler/simulator to generate it).

Member 4 — Sagda Hossameldin Ibrahim

Role: Top-Level Integration & PC/MUX/Branch Logic

5.11 Program Counter (PC)

Signal	Dir	Width	Description
clk	in	1	clock

rst	in	1	synchronous reset
pc_next	in	32	next PC value (from MUX)
pc_out	out	32	current PC

5.12 Top-Level Datapath

Instantiates and wires all modules above, plus the ALUSrc / MemtoReg / PC-source multiplexers and the branch-target adder/comparator. Owns the overall clk/rst ports and the memory-mapped I/O output port used for the demo.

5.13 Tasks

- Implement the PC register, PC+4 adder, branch-target adder, and the PC-source / ALUSrc / MemtoReg MUXes.
- Implement branch comparator logic (BEQ/BNE) combined with the Branch control signal.
- Integrate all members' modules into the top-level datapath on Day 4; own the reset/clock strategy and any timing fixes.

Member 5 — Haroun Taha

Role: Testbench, Assembly Programs, I/O & Documentation

5.14 I/O Interface

Signal	Dir	Width	Description
reg_data	in	32	value of a designated register or memory address to display
leds	out	8	or 7-seg segments, mapped to reg_data[7:0]

5.15 Tasks

- Write 3 assembly test programs: (1) arithmetic sequence, (2) counting loop using BNE, (3) memory store/load round-trip (see Section 6).
- Write the top-level integration testbench that runs each program and checks the final register/memory state.
- Implement the I/O interface module for the FPGA demo (LEDs / 7-segment).
- Own the final report and presentation slides; coordinate the Day 5 demo script.

6. Assembly Test Programs

Program 1 — Arithmetic

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10
add  x3, x1, x2     # x3 = 15
sub  x4, x2, x1     # x4 = 5
```

Program 2 — Loop with Branch

```
addi x5, x0, 0      # counter = 0
addi x6, x0, 5      # limit = 5
loop:
addi x5, x5, 1
bne  x5, x6, loop
```

Program 3 — Memory Store/Load

```
addi x7, x0, 100    # value = 100
sw   x7, 0(x0)      # MEM[0] = 100
lw   x8, 0(x0)      # x8 = MEM[0] = 100
```

7. Integration Checklist (Day 4)

- All individual module testbenches pass.
- Top-level module wires match the port tables in Section 5 exactly (no signal name/width mismatches).
- Run Program 1, 2, and 3; check final register values via waveform or $display.
- Fix any control-signal or timing mismatches before moving to synthesis.

8. Risk & Time-Saving Notes

- Do NOT attempt pipelining or the full RV32I set — single-cycle plus a restricted subset is what makes the 5-day deadline realistic.
- Agree on port names/widths (Section 5) before anyone writes code — this is what prevents Day 4 integration delays.
- Keep instruction/data memory small (e.g., 64 words) to simplify indexing and $readmemh loading.
- If FPGA synthesis time runs short on Day 5, simulation waveforms plus a partial hardware demo are still an acceptable fallback — prioritize having a fully verified simulation first.

9. Expected Deliverables

- Complete, synthesizable Verilog RTL for the processor and all sub-modules.
- Testbenches and simulation waveforms proving correct instruction execution.
- FPGA bitstream and a live hardware demo.
- Final technical report and presentation slides.